

Proof-of-Context: The Missing Verification Layer in Decentralized ML Protocols

Juan Cruz Maisu
Buenos Aires, Argentina
juancmais@outlook.com · github.com/Jc-asastu

Version 0.9.1 — Published: 21 June 2026 (v0.9; patch v0.9.1 of 5 July 2026). v0.9 adds **§9.6 + Corollary 1** — multi-party FGSS soundness, the dark-pool gate G^N proven sound by a per-party union bound over Theorem 1 — and a **§10.6 Evaluation** with measured costs: a full multi-party dark-pool settlement consumes 120,799 compute units ($\approx 8.6\%$ of Solana’s per-tx budget), the off-chain oracle paths run in 1–53 μ s, the suites pass 84 (crate) + 96 (on-chain) tests, and the canonical-JSON roots converge byte-identically across Rust/JS/TS. v0.9.1 corrects stale in-text version references and records the resolution of the §10.2 **Other**-tag non-injectivity in crate v0.3.1. No prior theorem is retracted; full history in the Revision note.

“PAL^{}M attests that a computation happened correctly; Proof-of-Context makes those attestations economically perishable — binding freshness to settlement so that stale inferences cannot clear payment.”*

Abstract

Decentralized machine-learning protocols have made significant progress on verification primitives — *proof-of-learning* in distributed training marketplaces (Gensyn [15]), *zkML proofs paired with TEE attestations* in verifiable-inference networks (OpenGradient [31]), *locality-sensitive hashing of inference activations* in production training-inference hybrids (Prime Intellect’s TOPLOC [16], deployed at 32B parameter scale in INTELLECT-2 [29]), and *property attestation* frameworks composing phases under centralized-provider threat models (PAL^{*}M [17]). Concurrent and prior work has also begun to touch specific facets of the problem we name: Verifiable Dropout [14] binds context strings to dropout randomness; CIV [18] supplies per-token provenance against prompt injection; Bittensor’s Commit-Reveal [19] uses staleness as cryptoeconomic offense against weight-copiers.

These contributions are fragment-level or differently-motivated instances of what this paper argues is a single gap — which we name **proof-of-context**. Existing primitives attest that a computation happened correctly. They do not make those attestations *economically perishable*. Proof-of-context does: it binds freshness to settlement so that a computation whose context has aged beyond a protocol-defined horizon cannot clear payment, regardless of its computational correctness.

We formalize four freshness types (computational, model, input, settlement), identify the execution-context root that any viable construction must commit to, position the primitive relative to existing work along the verification-vs-settlement axis, and sketch construction constraints — including the honest threat model of the triple-anchor defense (three clocks with orthogonal failure physics, defending against accidental skew only under a valid TEE attestation chain). The gap is the structural analogue, in ML protocols, of the oracle-freshness problem that accounts for significant documented value loss across decentralized-finance protocols [13]. This revision adds a formal model of the primitive, defines *freshness-gated settlement soundness*, and proves it for the subset our reference implementation enforces under standard cryptographic

assumptions, a valid TEE attestation chain, and an explicit honest-clock assumption that attestation alone does not supply; the model- and input-freshness guarantees reduce, precisely, to an oracle-freshness assumption — the same discipline the DeFi analogy imports. A reference crate realizes the model with real SHA-256 and Ed25519 cryptography and a passing end-to-end test suite.

Keywords: proof-of-context · decentralized machine learning · oracle freshness · settlement gating · prompt-cache integrity · cryptoeconomic design

1 Introduction

The emerging class of decentralized ML protocols — training marketplaces, verifiable-inference networks, agent-compute exchanges — have inherited a specific verification posture from their cryptographic roots: *prove that the computation happened, prove that it produced the claimed output*. Proof-of-learning does this for training steps. zkML proofs do this for inference. TEE attestations do this for hardware execution. Refereed delegation (Verde [15]) does it for general ML program execution. Locality-sensitive activation hashing (TOPLOC [16]) does it for inference integrity at production scale. All five are real, load-bearing, and hard-won.

This paper argues that this posture, while necessary, is *not sufficient for decentralized economic settlement*. Verifying that a computation was executed correctly is a statement about the past — an attestation with no price and no expiration date. But in a decentralized marketplace where workers are paid for inferences whose value depends on timing (a trading decision, a risk score, a routing choice, an oracle update), a correctness attestation that does not expire is an open grieving surface. The gap is not that existing primitives fail to verify computation. The gap is that they produce statements with no *economic perishability*.

We name this gap **proof-of-context**: a composable verification-primitive layer that sits on top of PoL / zkML / TEE / refereed delegation / activation hashing, binds the computation to a commitment over its runtime context state, indexes that commitment against a protocol-defined freshness horizon, and gates settlement against the commitment. If the context has aged beyond horizon, the protocol does not pay. The underlying math-correctness primitive still holds; the economic claim no longer does.

Our contributions are:

1. **The name** for the unified phenomenon — *proof-of-context*.
2. **The distinguishing axis** from closest prior art PAL*M [17]: attestation-as-verification (PAL*M) vs. attestation-as-settlement (PoC). See §3.6.
3. **The cross-phase unification**: training-state, inference-state, and prompt-cache-state freshness treated as facets of a single primitive, under a decentralized-marketplace threat model.
4. **The cross-domain analogy** to the oracle-freshness problem in decentralized finance [12, 13].
5. **Four-type freshness decomposition** (computational f_c , model f_m , input f_i , settlement f_s) as the primitive’s state variables. See §6.
6. **Execution-context root** formalization — the scope of what a PoC commitment must bind to avoid trivial evasion vectors. See §8.
7. **Composability framing** — PoC admits Verifiable Dropout, TOPLOC, Verde, Commit-Reveal, CIV as fragment-level instances.

8. **A formal model and soundness theorem** (§9–§10) — *freshness-gated settlement soundness*, proved for the enforced subset under collision-resistance, EUF-CMA, attestation soundness, and an explicit honest-clock assumption, with the model- and input-freshness guarantees reduced to an explicit oracle-freshness assumption, and realized in a reference crate with real cryptography and a passing test suite.

We are not the first to observe that context — specifically model staleness, asynchronous coordination, nondeterminism, inference integrity — matters in decentralized ML. A substantial literature in federated learning has studied these problems for half a decade [5–9]. Our contribution is neither the discovery of the problem nor a novel cryptographic primitive. It is the *protocol framing*: a composable settlement layer that imports DeFi’s hard-won oracle-freshness discipline into the ML verification stack.

The motivating observation is cross-domain. In decentralized finance, the analogous gap — the oracle-freshness problem — has been responsible for documented value losses across every major AMM, lending protocol, and derivatives platform, surveyed systematically in [13]. Systems that verified “did the swap execute correctly?” failed because they did not verify “was the price you swapped against fresh?” We expect the same pattern to appear in decentralized ML, and argue it is already latent in production systems.

2 Background: Existing Verification Primitives

We briefly survey five primitives currently deployed in decentralized ML protocols.

Proof-of-learning. In protocols such as Gensyn, a worker commits on-chain to intermediate gradient checkpoints at agreed intervals. Verifiers re-run short segments of the training step against the submitted checkpoints and confirm that the gradients match. The guarantee: *the reported gradient was produced by the claimed training operation on the claimed input*. The foundational construction was introduced by Jia et al. [1]. Subsequent work has documented spoofability under certain conditions [2, 10] and sensitivity to hardware nondeterminism [3].

zkML proofs. In protocols such as OpenGradient, a worker produces a zero-knowledge proof that a specific model, identified by its weight commitment, was executed on a specific input and produced a specific output. The guarantee: *the reported inference is cryptographically bound to the claimed model + input pair*. The field is surveyed in [4, 11].

TEE attestations. A Trusted Execution Environment produces a signed attestation that the computation occurred inside the enclave, that the enclave ran the claimed code, and that the output was produced by that code. The guarantee is relative to the hardware trust root (Intel TDX, AMD SEV, Nvidia H100 confidential compute). TEE attestation is the primitive that anchors PAL^{*}M’s [17] runtime claims and is a defensive layer against enclave *code* compromise; its role in this paper’s construction sketch is discussed in §7.

Refereed delegation with deterministic replay. Gensyn’s Verde construction [15] pairs an optimistic refereed-delegation scheme (multiple providers re-execute; disagreement triggers bisection down to a single operation) with a library (RepOps) enforcing bitwise reproducibility across heterogeneous GPU hardware. The guarantee: *the reported ML program execution is bitwise-verifiable against a single honest replica*. The construction explicitly scopes out cache behavior, data-oracle freshness, context-dependent validity, and sampling-algorithm fidelity.

Inference-activation LSH fingerprinting. Prime Intellect’s TOPLOC [16] hashes the last-layer hidden-state activations using a top-k polynomial-encoding scheme (258 bytes per 32 tokens, $\sim 1024\times$ reduction), achieving 0% false-positive and 0% false-negative rates against model replacement, prompt modification, and precision downgrade in its empirical regime. It is the closest production primitive to context-binding: a single hash couples prompt, model weights, and precision into one verifiable fingerprint. Scope is per-inference, post-hoc. TOPLOC’s authors explicitly document three uncovered surfaces: speculative decoding (by design), FP8-vs-bf16 precision-boundary cases, KV-cache compression (untested), and adversarial “unstable prompt mining” [16, §6].

Each of these primitives verifies a dimension of computational correctness. Together they cover much of the attack surface a malicious worker might exploit to fabricate work. What they do not cover — individually or in combination as currently deployed — is the question of whether settlement should occur: whether the correct computation, executed on the correct input, was performed under a contextual state that has not aged past the protocol’s economic horizon.

3 Related Work

Our contribution sits at the intersection of several existing threads. We organize by overlap severity.

3.1 Staleness-aware federated and decentralized learning. A substantial body of work in asynchronous federated learning has studied the effect of model staleness on training quality [5, 6, 7, 8]. Techniques include staleness-weighted gradient aggregation, temperature-controlled weighting, dynamic staleness control, and behavioral staleness metrics [6, 7, 8]. The decentralized-blockchain extension [9] quantified accuracy degradations of up to $\sim 35\%$ attributable to staleness and inconsistency. Recent work on lightweight blockchain for verifiable federated learning on edge networks [23] covers the training half of our frame but does not name the primitive, does not address inference or prompt-cache, and does not import the DeFi-oracle analogy. The broader tradition treats staleness as a *coordination and weighting* problem. Our framing shifts the question to whether contextual appropriateness can be *verified at contribution time as a primitive*, and *refused* rather than down-weighted.

3.2 Proof-of-learning robustness. The foundational construction of Jia et al. [1] was shown to be spoofable by subsequent analyses [2] and a follow-up by the original research group [10]. Optimistic Verifiable Training [3] addresses nondeterminism specifically. The concern is orthogonal to ours: hardware nondeterminism is about whether the *same* computation can be reproduced bit-exactly, whereas proof-of-context is about whether a correctly-performed computation should *clear settlement* given the age of its contextual state.

3.3 Verifiable-ML lifecycle taxonomies. Recent surveys [4, 11] and the SoK of Bruschi et al. [20] partition ML verification into training, testing, inference, and (for FL) aggregation. The SoK’s phase-based decomposition — Proof of Committed Data, Proof of Training, Proof of Aggregation — is the scaffold we extend. Proof-of-Context enters this scaffold as a fourth phase covering the post-training inference lifecycle, which the SoK explicitly excludes [20, §1.1]. A related framework [21] frames training+inference+unlearning as an end-to-end pipeline and enumerates its components; our contribution names the binding primitive they enumerate.

3.4 Context-binding in stochastic operations. Verifiable Dropout [14] is the nearest construction on the binding-mechanism axis. It binds dropout masks to a context payload (`model_id`,

step, batch_id, verifier-nonce, layer_id) via a hash-and-sign chain (SHA256 + Ed25519) and proves the result in a zkVM (RISC Zero). This is exactly the shape of context-binding payload we propose to generalize. The scope of [14] is strictly training-side dropout; it never addresses inference, cross-phase, freshness, staleness, oracle semantics, or settlement gating.

3.5 Inference integrity via locality-sensitive hashing. TOPLOC [16] is the production-grade inference-side construction (0% FPR, 0% FNR; 258 bytes per 32 tokens; 0.26 ms/token commit overhead; fixed dual thresholds for bf16: $T_{exp}=38$, $T_{mean}=10$, $T_{median}=8$). TOPLOC authors’ explicit acknowledgment that attention implementation, precision mode, and KV-cache compression are attack-surface vectors is the basis for our execution-context-root scope (§8); we attribute the identification of these vectors to TOPLOC and cite accordingly wherever they appear. The construction does not carry freshness semantics: each commitment is per-rollout and post-hoc. Our framing positions TOPLOC as an inference-integrity fragment of the proof-of-context primitive and makes KV-cache and prompt-mining surfaces first-class rather than acknowledged-but-uncovered.

3.6 Property attestation across phases (closest prior work on composability). PAL^{*M} [17] is the nearest prior work on cross-phase-composability. It composes “proof of training,” “proof of inference,” and “proof of session inference” in a single property-attestation framework under a centralized-provider threat model (TDX + H100 confidential compute), with a challenge-response “Chal” device for anti-replay freshness. **The closest-surface overlap is high enough that explicit distinction is required.**

PAL^{*M} and proof-of-context differ fundamentally on the *attestation vs. settlement* axis:

PAL^{*M} is attestation-as-verification. Its output is a statement: “this model, from this provider, has these properties; this inference session ran on that model with those parameters.” The statement is a truth about the past. It has no price. It does not expire. Its purpose is to allow a regulator, auditor, or compliance-bound customer to verify ex post that the provider did what they claimed. The Chal device prevents a provider from caching a single favorable attestation and replaying it to many verifiers — but an individual attestation, once issued, is a permanent record of what happened.

Proof-of-context is attestation-as-settlement. Its output is a gated claim: “this computation is eligible for payment if and only if its context commitment is within freshness horizon K of the current canonical global state at the moment the payment clears.” The claim is economically perishable: it ages, and at some point it stops being a valid settlement instrument. Its purpose is to allow a decentralized marketplace to refuse payment for work whose contextual state has drifted out of the economically-useful window, without requiring trusted providers, post-hoc auditors, or off-chain dispute.

This distinction is not rhetorical. The two primitives operate at different positions in the computation lifecycle, have different threat models (centralized provider vs. decentralized marketplace), have different freshness semantics (anti-replay vs. temporal-validity gating), and fail in different ways (PAL^{*M} fails when the regulator cannot verify the attestation; PoC fails when stale computations clear payment). Neither work uses the term *proof-of-context*; PAL^{*M} does not import the DeFi oracle analogy; PoC does not replace PAL^{*M} for regulatory-audit use cases. The two are complementary primitives targeting distinct failure modes of decentralized ML.

3.7 Per-token contextual integrity. CIV [18] supplies per-token HMAC provenance labels with a trust-lattice attention mask, defending against prompt-injection attacks within a single inference

call. CIV’s threat model is within-context injection; proof-of-context’s threat model is stale-context substitution between calls. CIV is a per-token integrity layer; proof-of-context is a per-session settlement layer. The two are orthogonal and potentially composable.

3.8 Tensor-native proof-of-inference. TensorCommitments [22] produces Merkle-style commitments over tensor states during inference with $\sim 1\%$ overhead on LLaMA2, under the name “proof-of-inference.” The name adjacency to “proof-of-context” is terminological: TensorCommitments is an inference-only commitment primitive without cross-phase, cache, or freshness semantics. It is a candidate lower-layer primitive that proof-of-context could compose over.

3.9 Staleness as cryptoeconomic offense. Bittensor’s Commit-Reveal [19] is the nearest cryptoeconomic precedent for staleness in decentralized ML. Validators commit BLAKE2(weights, nonce) on-chain during a commit window; weights are revealed after a `commit_reveal_period` (default one tempo ≈ 72 minutes on mainnet). Staleness-copying validators depart from Yuma consensus and suffer emergent dividend penalties proportional to the MSE between their (stale) reported weights and the (current) consensus. The mechanism is *producer-side* — it hides producer signals from peer producers to deter consensus-copying. It has no consumer-side freshness semantic: downstream users of the subnet’s outputs receive no cryptographic recency guarantee. Proof-of-context occupies the complementary axis: consumer-side settlement gating via cryptographic temporal-validity, not producer-side signal hiding via cryptoeconomic dividend decay.

3.10 Oracle freshness in DeFi. The oracle problem in decentralized finance is not typically discussed in the ML literature, but it is directly relevant and underlies the analogy at the center of this paper. The use of time-weighted average prices (TWAPs), multi-source quorums, freshness thresholds, and per-block price deviation checks emerged as cryptoeconomic responses to what was initially treated as a data-quality concern — and is now understood as a verification-layer concern [12]. The SoK of Zhou et al. [13] surveys DeFi attack vectors systematically and positions oracle-freshness failures prominently. A recent exploration asks whether AI can *solve* the blockchain oracle problem [25]; our analogy runs in the opposite direction, importing DeFi’s oracle-freshness discipline into ML verification.

3.11 Adjacent concerns we flag but do not unify. The RAG-security literature [24] identifies corpus-freshness and provenance-tracking as open problems. Enterprise governance frames such as Context Kubernetes [26] introduce freshness-aware state machines (fresh/stale/expired/conflicted) but in an orchestration rather than cryptographic register. Right to History [27] proposes verifiable agent-execution ledgers. These are related but distinct concerns — none is a cryptographic context-verification primitive gating settlement under a decentralized threat model.

Summary. The literature has surfaced most of the pieces — staleness concerns, spoofability gaps, nondeterminism controls, phase-specific verification primitives, inference-activation integrity, centralized-provider property attestation, within-context integrity, cryptoeconomic-offensive staleness mechanisms. It has not, to our knowledge, named the unification on the attestation-as-settlement axis, proposed the DeFi-oracle-freshness analogy for ML, extended the framing to prompt-cache integrity, or framed the problem as requiring a composable settlement primitive at the decentralized-marketplace protocol layer rather than better scheduling, stronger centralized attestations, or individually-hardened existing primitives.

4 The Gap: Computational Correctness vs. Economic Perishability

Machine-learning training and inference, unlike a token swap, are not memoryless. Every gradient step depends on the global state of the model at that moment. Every inference call depends on the current model version, prompt cache, system prompt, tool bindings, and oracle feeds it may have consumed. The computation is only meaningful in the context in which it was performed.

A worker can produce a mathematically correct gradient on a stale model snapshot. The gradient passes proof-of-learning verification trivially. But aggregated into the global optimizer, it actively degrades the training run. The staleness-aware FL literature addresses this by *weighting* the gradient down; we argue the protocol layer should additionally have the option of *refusing to settle payment* for gradients below a contextual-freshness threshold, as a settlement primitive rather than a post-hoc coefficient.

A worker can produce a valid zkML proof of an inference call on a stale model version. The proof passes verification trivially. But the client received an answer from a deprecated model. No equivalent of the staleness-aware-FL tradition exists on the inference side yet, which is partly why this framing is needed.

Production inference-integrity systems such as TOPLOC achieve strong guarantees against model replacement, prompt modification, and precision downgrade (0% FPR/FNR in their empirical regime [16]), yet their authors explicitly flag speculative decoding, FP8 precision boundaries, KV-cache compression, and adversarial “unstable prompt mining” as uncovered attack surfaces [16, §6]. All four sit inside the settlement-gating frame we propose: they are failures of contextual validity at settlement time rather than computational correctness at generation time.

The generalization: **a computation can be correct and still not be worth settling**, if the context under which it was performed has drifted out of the economically-useful window the protocol defined.

5 The DeFi Analogy

Decentralized finance has lived with exactly this gap for six years.

Early AMMs verified that swaps executed according to their bonding curves. The swap itself was always correct: given a pool state, the curve produced the deterministically correct output amount. What AMMs did not verify — and what flash-loan attackers exploited repeatedly — was whether the *pool state itself* reflected fresh price information at the moment of the swap. Attackers temporarily manipulated the pool state (via flash loans), swapped against the manipulated state (with computational correctness), and exited. Every swap passed the protocol’s verification primitive. Documented value losses across these attack classes are catalogued systematically in Zhou et al.’s SoK on DeFi attacks [13].

Lending protocols tracking collateral value via on-chain oracles suffered the same pattern under the name *stale oracle exploits*. The price feed was occasionally late by one block or one epoch, and attackers used the delay to liquidate healthy positions or borrow against undervalued collateral. The feed was never wrong in the sense of reporting a fabricated number. It was wrong in the sense of reporting a true number *from the wrong moment*.

The DeFi response to this gap, over several years, was the construction of a full secondary verification layer around *context verification* — freshness, deviation thresholds, time-weighted averages, multiple-source quorums, circuit breakers [12, 13]. The primary verification of computational correctness stayed where it was. But on top of it, the industry built a settlement layer that became load-bearing for safety: a price feed is not merely a number you trust, it is a number you *refuse to*

settle against if its freshness attestation has expired.

Claim. The decentralized-ML protocol stack is, in 2026, roughly where DeFi was in 2020 on this axis. The primary verification layer (PoL, zkML, TEE, refereed delegation, inference-activation LSH) is hardening quickly; its robustness concerns are well-studied [1–3, 10, 15, 16]. The staleness-weighting literature [5–9] addresses context as a scheduling concern. Centralized-provider property attestation [17] composes phases but under a different threat model. A settlement-gating primitive at the decentralized-protocol level, composing across training, inference, and prompt-cache, with cryptographic freshness semantics imported from DeFi oracle design, does not yet exist in the literature.

6 Four Freshness Types

A first-class treatment of proof-of-context requires decomposing freshness into four distinct types. Each fails differently; each requires different measurement and enforcement; and each maps to a subset of the failure modes named in prior work.

f_c — **Computational freshness.** The elapsed time between when a computation was performed and when its attestation is submitted for settlement. High f_c means the worker sat on the result. Measured in protocol time (block height, TEE timestamp, external beacon round — see §7).

f_m — **Model freshness.** The distance between the model version the worker used and the canonical on-chain model version at settlement time. High f_m means the worker used a stale model snapshot. Measured in version-epoch distance (number of root bumps between used version and current version).

f_i — **Input freshness.** The temporal validity of input-world state consumed by the computation: oracle feed values, RAG retrieval corpus version, tool-call result timestamps, prompt-cache entries. High f_i means the worker’s inputs were taken from stale sources even if the model was current. This is the axis most economically critical for agent-inference use cases (DeFi routing, risk scoring, real-time trading advice) — the one the existing federated-learning literature does not address because it predates agentic inference patterns.

f_s — **Settlement freshness.** The permitted window between attestation commit and settlement clearance. High f_s tolerance allows batched settlement and dispute delay at the cost of permitting workers to sit on commits that were fresh at commit time but stale at settlement time. Low f_s tolerance forces tight clearing but excludes legitimate use cases with built-in delay (refereed-delegation challenges, optimistic dispute windows).

Four failure modes, rebuilt around these types:

F1. f_m violation — stale model snapshot. A worker computes gradients (training side) or inference (inference side) against a model version multiple root bumps behind. Corresponds to the classical staleness-aware-FL mode [5, 9] on the training side, and to the “deprecated-model inference” mode on the inference side. TOPLOC [16] detects model replacement per-rollout but does not gate settlement on f_m .

F2. f_i violation — stale input-world state. A worker feeds the computation with oracle values, RAG content, tool-call results, or prompt-cache entries older than the protocol’s f_i horizon. The computation is mathematically correct against the inputs received. It is economically invalid because those inputs no longer represent the world state at settlement time. This failure mode has no direct analogue in the federated-learning staleness literature. It is the one with highest economic weight for agent-inference use cases and the one TOPLOC’s “unstable prompt mining” attack surface [16, §6] partially touches.

F3. f_c violation — worker sat on the result. A worker performs a valid computation against fresh state but delays submission to the settlement layer. By submission time, the world has moved; paying the worker rewards effectively retrospective reasoning. Typically bounded by a protocol-defined `max_f_c` after which submissions are rejected or down-priced.

F4. f_s violation — computation was fresh at commit, stale at clear. A worker commits honestly against current state but exploits a large settlement window to delay clearing until the protocol has moved past the relevant decision point. Typically bounded by `max_f_s` in tension with legitimate dispute windows (§7 constraint 4).

All four are invisible to PoL, zkML, TEE, and refereed delegation in their current forms — those primitives answer “*was the computation correct?*” not “*is it still valid to settle on?*” TOPLOC partially covers F1 (inference side) and F2 (prompt modification but not KV-cache substitution) as a post-hoc per-rollout hash. Verifiable Dropout partially covers an adjacent aspect of F1 for single stochastic operations. PAL^{*}M covers F1 and F2 attestation concerns for centralized providers but does not gate settlement in a decentralized marketplace. None covers all four freshness types as a composable settlement-gating primitive.

7 A Research Program — Construction Constraints

We do not in this paper propose a complete cryptographic construction for proof-of-context. That construction is the central open research question the framing invites. We sketch the constraints any viable construction must satisfy, informed by the Phase 2 literature survey and by the critiques this paper absorbed during drafting.

1. **Freshness is verifiable per-computation, not reputational.** The protocol must reject a specific gradient / inference / contribution for contextual staleness without relying on long-horizon reputation scores. Cf. the DeFi evolution away from “trusted reporter” oracle models toward per-query freshness proofs.
2. **Context commitments must be cheap.** If committing to the contextual state of a training step costs more than the training step itself, the economics fail. The construction likely requires Merkle commitments, hash chains, or amortizable structures on top of existing PoL / zkML / TEE / LSH primitives, not parallel to them. TOPLOC’s 258 bytes per 32 tokens sets a useful cost benchmark for the inference side [16].
3. **Cross-worker consistency is required.** Training protocols cannot verify contextual appropriateness by asking a single worker to report its own context. The construction must cross-reference context claims across a worker set or against a canonical global state published on-chain.

4. **Graceful degradation along each freshness type.** Binary pass/fail schemes will reject too many honest workers whose f_c , f_i , f_m , or f_s is slightly above horizon for innocuous reasons. Continuous scoring schemes collapse into reputation in disguise. A *tolerance bound per freshness type*, expressed in protocol-agnostic terms, is likely a required primitive.

5. **Compatibility with existing verification primitives, under explicit composition rules.**

Proof-of-context is a layer *on top of* PoL / zkML / TEE / refereed delegation / LSH fingerprinting, not a replacement.

- Extends the context-payload shape used by Verifiable Dropout [14] from seed-binding for a single stochastic operation to first-class commitment over per-computation state.
- Composes with the activation-integrity primitive of TOPLOC [16]: a PoC commitment may embed or reference a TOPLOC-compatible hash when one is available.
- Admits the refereed-delegation primitive of Verde [15] to accept freshness-bound challenges alongside math-correctness challenges.
- Inverts the staleness treatment of Bittensor Commit-Reveal [19] from producer-side hiding to consumer-side freshness gating.
- Coordinates with root-bump mechanics using **prospective-only semantics**: a publisher bump of the execution-context root from t to $t+1$ does *not* invalidate attestations already committed against t . Workers with in-flight commitments settle against t within their settlement window; new commitments after the bump must reference $t+1$. This protects workers from retroactive grieving without requiring a publisher bond.
- Requires a **runtime-upgrade notice period**: when a bump changes runtime requirements (new CUDA, more VRAM, larger model) that would exclude previously-eligible workers, the publisher must announce N blocks ahead of activation so workers can exit without loss.
- Requires **historical weight storage** during open settlement windows: the publisher must keep weights associated with root t available until all settlement windows against t have closed. If the data-availability layer fails to serve, the default resolution is *favor the worker* — a worker with a valid signed commitment cannot lose stake because a publisher’s operational failure prevented post-hoc verification.

6. **Triple-anchor-or-slash — three clocks with orthogonal failure physics, honest about threat model.**

Temporal validity in the protocol is anchored to three independent clocks:

- **Block height** (chain-local clock, vulnerable to MEV, reordering, and chain reorg).
- **TEE timestamp** (enclave-local clock, vulnerable to manipulation from inside a compromised enclave — SGX clock attacks are documented in the TEE security literature).
- **Drand round** (external threshold-BLS clock at 30-second granularity, vulnerable to compromise of 2/3 of the League of Entropy threshold).

A commitment includes all three. Divergence between them beyond expected skew is cause for slash.

Honest claim about what this protects against: under the assumption of a **valid TEE attestation chain** (TDX quote + H100 attestation report verified on-chain against known-good measurements), the triple-anchor protects against (a) accidental skew between honest

clocks under network load and (b) isolated failure of a single clock (chain reorg, Drand network hiccup, local TEE clock jitter).

What the triple-anchor does NOT protect against: a compromised TEE that observes block height and Drand round at signing time and simply echoes them as its own timestamp. The enclave is downstream of the other two clocks; an adversary inside the enclave can always report a timestamp consistent with the external ones. Defense against enclave *code* compromise is the attestation chain; a *timing-channel* compromise of an otherwise-attested enclave (the TDXdown class) is caught by neither the triple-anchor nor attestation — it is carried as the explicit honest-clock assumption (§9.4, (H3b)). This must be said plainly because the claim that three clocks protect against any adversary breaks under the first technical review.

Expected skew parameter calibration is empirical, not handwave: the thresholds must be measured against (i) Base block-time distribution, (ii) documented SGX/TDX clock drift from the TEE security literature, and (iii) Drand round arrival latency distribution across relay networks. The experimental calibration and proposed default thresholds are in §11.

7. **Data-availability layer decision.** Historical weight storage during settlement windows, dispute-resolution artifacts, and long-tail attestation records require a DA layer. EigenDA (via EigenLayer AVS) is the tentative pick: alignment with restaking-AVS deployment pattern, existing precedent in the ecosystem, and cost model compatible with weight archival. Celestia and Base blob storage are alternatives. Final selection is pending empirical cost analysis and is the primary architectural decision before crate scaffold.
8. **Freshness semantics borrowed explicitly from DeFi.** The construction imports the cryptoeconomic disciplines that DeFi hardened over six years: heartbeat (every commitment carries a timestamp no older than a protocol-defined horizon), deviation threshold (a commitment may be valid even if stale provided state-distance from canonical is below a bound), multi-source quorum (critical context such as model version attested by a committee), and circuit breakers (sudden deviation triggers protocol-level halting). The specific insight that *freshness must be verified at pay-time rather than discovered retroactively* — which this constraint formalizes — derives directly from the author’s production work in BaseOracle (see §12).

8 Execution-Context Root

A proof-of-context commitment is keyed to an *execution-context root* — a Merkle commitment over every protocol-relevant component of the execution environment. Defining the scope of this root correctly is load-bearing: any component affecting output that is *not* in the root is a trivial evasion vector.

Minimum scope for the execution-context root (as a Merkle commitment over these elements):

- `weights_hash` — model weights at commit time
- `tokenizer_hash` — tokenizer identity and version
- `system_prompt_hash` — system prompt for the inference session
- `sampling_params` — temperature, top_k, top_p, seed

- `runtime_version` — CUDA, cuDNN, driver, inference runtime (vllm, sglang, etc.)
- `attention_impl_id` — FlashAttention vs SDPA vs Flex; identified by TOPLOC [16] as an attack-surface vector
- `precision_mode` — fp16, bf16, fp8; identified by TOPLOC [16] as an attack-surface vector
- `inference_config` — `max_tokens`, stop sequences, penalty parameters
- `input_manifest_root` — Merkle root over input sources: oracle IDs, RAG corpus version, tool bindings (this is the channel through which f_i freshness is anchored)
- `kv_cache_root` — when applicable, Merkle root over KV-cache state

What happens if scope is wrong: a worker changes any in-scope component and produces a different output but the root does not move; PoC commitment passes; settlement clears. Every in-scope component must be present at the granularity at which it affects output.

Attribution: `attention_impl_id` and `precision_mode` as components of the root derive directly from the attack-surface analysis published by Prime Intellect’s TOPLOC team [16]; they would not be in the root without that work.

What the root excludes by design:

- Anything the worker may legitimately choose at inference time that does not affect protocol economics (e.g., internal logging, debug verbosity).
- Anything guaranteed bit-exact by refereed-delegation primitives [15] and thus redundant to commit separately.

Open problem: formally proving that the scope above is sufficient to eliminate all economically-relevant undefined-behavior vectors, for a given class of inference runtimes, is ongoing work. This is probably half the work of any mature proof-of-context construction.

9 Formal Model and Soundness

Sections 6 through 8 describe proof-of-context informally: four freshness types, a set of construction constraints, and the scope of the execution-context root. This section makes the core precise enough to state and prove a soundness property. We deliberately formalize only what the reference implementation (§10) enforces, and we are explicit about where each guarantee’s trust is located. The model is small; its value is that it names the exact assumptions under which a stale computation cannot clear payment, and the exact point at which each assumption, if removed, breaks the guarantee.

9.1 Objects. Let H denote SHA-256 and $\parallel$ concatenation in a fixed canonical field order (the order is part of the protocol; changing it is a breaking change).

Execution-context root. The root is a hash commitment over the ten components enumerated in §8:

$$R = H(\text{weights} \parallel \text{tokenizer} \parallel \text{system_prompt} \parallel \text{enc}(\text{sampling}) \parallel \text{runtime} \parallel \text{attn} \\ \parallel \text{precision} \parallel \text{enc}(\text{inference_config}) \parallel \text{input_manifest} \parallel [\text{kv_cache}]),$$

where $\text{enc}(\cdot)$ is the fixed-width encoding of the structured fields and the kv-cache component is present only when the inference uses a persistent cache. The canonical preimage is 251 bytes without

the kv-cache and 283 with it (one byte more in each case when the attention implementation is an **Other** tag, whose injective encoding is the two-byte sequence `0x80 || tag` — see §10.2).

Triple anchor. A commit-time clock reading $A = (h, \tau, \rho)$: block height h (u64), TEE timestamp τ (u128), Drand round ρ (u64), so the fixed-width encoding $\text{enc}(A)$ is $8 + 16 + 8 = 32$ bytes. We write $\text{skew}(A, A') = (|h - h'|, |\tau - \tau'|, |\rho - \rho'|)$, and say A *diverges* from A' beyond a threshold vector θ , written $\text{diverge}_\theta(A, A')$, when any skew component exceeds its bound in θ .

Commitment. A commitment is $C = (R, A, y, \sigma, pk, \chi)$, where y is the output hash, σ is an Ed25519 signature over the signing digest $d = H(R || \text{enc}(A) || y)$, pk is the verifying key, and χ is a TEE attestation chain. The signature binds root, anchor, and output into one object; no in-scope field can change without invalidating σ .

Thresholds. A vector θ fixes, per deployment, the three skew bounds and the four freshness horizons (`max_fc`, `max_fm`, `max_fi`, `max_fs`). The empirically-justified Base-mainnet defaults are given in §11.

9.2 Freshness predicates and the settlement gate. For a commitment C , a settlement-time anchor **now**, and thresholds θ , we define one anchor-integrity predicate and four freshness predicates (one per freshness type):

- $\text{consistent}(C, \theta)$: the commit anchor’s three clocks corroborate *each other* within skew — the block-derived, TEE, and Drand-derived wall-times of A mutually agree. This is an *integrity* check on the anchor **alone** (a function of A , not of A versus **now**): it detects a tampered or desynced commit clock and, crucially, forecloses a worker presenting a block height out of line with its TEE and Drand readings. It is distinct from — and does not by itself bound — the staleness horizons below.
- $\text{fresh}_c(C, \theta)$: the computation was not held back between completion and commitment. This is the one type **not decidable from** (A, now) : $\text{now}.h - A.h$ is the commit→settle gap (that is fresh_s), not the compute→commit gap, which would require a computation-completed timestamp the commitment does not carry. We therefore treat f_c *structurally* (commit-at-completion; see §10.2) rather than as a gate predicate, and the gate below omits it.
- $\text{fresh}_s(C, \text{now}, \theta)$: the settlement window has not elapsed, $\text{now}.h - A.h \leq \theta.\text{max_fs}$.
- $\text{fresh}_m(C, \text{now}, \theta)$: the committed model root is within $\theta.\text{max_fm}$ version-epochs of the canonical model root at **now**.
- $\text{fresh}_i(C, \text{now}, \theta)$: the committed input manifest lags the canonical input-world state at **now** by at most $\theta.\text{max_fi}$.

A settlement gate is the predicate $G(C, \text{now}, \theta) = \text{Verify}(C) \wedge \text{consistent} \wedge \text{fresh}_m \wedge \text{fresh}_i \wedge \text{fresh}_s$, where $\text{Verify}(C)$ checks the Ed25519 signature over d and validates the attestation chain χ . When G holds the gate returns **Clear**; otherwise it returns the set of violated predicates, so the protocol can emit the corresponding refund or slash event. (f_c is absent by design — handled structurally per §10.2, not gated.) Among the freshness predicates only fresh_m and fresh_i require state external to the commitment — a canonical model root and a canonical input-world state at settlement time; the integrity check consistent is decidable from C alone and fresh_s from C and **now**. We isolate the external dependency deliberately: it is the formal residue of the DeFi oracle-freshness discipline imported in §5.

9.3 Security definition. We model a malicious worker as a probabilistic polynomial-time adversary $\mathcal{A}$ that runs any computation it likes and then attempts to clear payment for a result whose context is no longer fresh. $\mathcal{A}$ obtains commitments through an honest commit oracle that, for each computation, records the genuine commit-time anchor A^* — the enclave’s clock reading at signing under (H3b) below — and returns the signed C ; $\mathcal{A}$ may also attempt to forge commitments directly.

Definition (Freshness-Gated Settlement Soundness, FGSS). *Fix θ . The adversary receives the public parameters and a settlement-time anchor now , and outputs a commitment C . $\mathcal{A}$ wins if $G(C, \text{now}, \theta) = \text{Clear}$ while the true context under which the committed output y was produced is stale relative to now — i.e., the genuine commit-time anchor A^* satisfies $\text{now}.h - A^*.h > \theta.\text{max_fs}$, the result was committed outside the settlement window (for the enforced $\{\text{consistent}, \text{fresh}_s\}$ subset; consistent additionally rejects any commitment whose anchor clocks do not mutually corroborate, foreclosing a fabricated-fresh block height), or the real model/input state exceeds the $\text{fresh}_m/\text{fresh}_i$ horizon (for the extended gate). Because Clear requires $\text{Verify}(C)$, which under (H3a) accepts only signatures from known-good enclaves, every commitment that could win was produced by an enclave that reported some reading A^* (whether that reading is truthful is the separate concern of (H3b)); so A^* is well-defined wherever the anchor-based win condition is evaluated (the $\text{fresh}_m/\text{fresh}_i$ extended conditions instead rest on canonical state under (H4)), and a commitment fabricated without an enclave signature fails Verify and cannot win. A gate is FGSS-sound if $\Pr[\mathcal{A} \text{ wins}] \leq \text{negl}(\lambda)$. The definition captures exactly the failure this paper is about: not “did the computation run?” but “should this run clear payment, given the world has moved?”*

9.4 Soundness of the enforced subset. We first state the property the reference implementation actually achieves, under four assumptions: **(H1)** H is collision-resistant; **(H2)** Ed25519 is existentially unforgeable under chosen-message attack; **(H3a)** *attestation soundness* — Verify accepts pk only when χ proves pk belongs to an enclave whose measurement is on the known-good list; **(H3b)** *honest enclave clock* — the attested enclave reports the genuine triple anchor at commit time. (H3b) is a trusted-clock assumption that attestation alone does **not** discharge: a correctly-attested enclave whose timing channel is compromised, as in the TDXdown attack discussed in §11, satisfies (H3a) yet violates (H3b). We keep the two separate for exactly that reason.

Theorem 1 (soundness of the enforced subset). *Under (H1), (H2), (H3a), and (H3b), the gate of §9.2 restricted to the predicates it enforces — the anchor-integrity check consistent and the settlement window fresh_s — is FGSS-sound. Concretely, if $G(C, \text{now}, \theta) = \text{Clear}$, then — except with probability negligible in λ , the collision/forgery slack of (H1) and (H2) — the commitment’s (R, A, y) is authentic; under (H3b) the committed anchor A is then the genuine reading A^* ; and the gate’s explicit checks force that A ’s three clocks mutually corroborate within skew (consistent) and that $\text{now}.h - A.h \leq \theta.\text{max_fs}$ (fresh_s). Because consistent ties the committed $A.h$ to the (honest, by (H3b)) TEE and Drand readings, a within-window $A.h$ cannot be a fabricated-fresh height; the staleness guarantee thus rests on fresh_s , with consistent foreclosing the fabricated-anchor evasion.*

Proof (sketch). (1) **Binding.** By (H1), and assuming the field encoding is injective (see §10.2 for one implementation caveat), R is a binding commitment to the ten context components and d to (R, A, y) ; an adversary clearing payment under a different context, anchor, or output exhibits a collision in H . (2) **Authenticity and truthfulness of the anchor.** A Clear requires $\text{Verify}(C)$, so σ is a valid signature over d under pk . By (H2) the signature cannot be forged without the key, and by (H3a) the key belongs to a known-good enclave, so the binding of A into d is authentic. The stronger claim $A = A^*$ — that the committed anchor is the *genuine* clock reading rather than one chosen to look fresh — is supplied by (H3b), not by EUF-CMA or attestation; it is the load-bearing

assumption of this step, and the one §9.5 removes to show where the guarantee ends. (3) **Gating**. Given $A = A^*$ and a **Clear**, the gate’s explicit checks force $\text{consistent}(A)$ — A ’s block-, TEE-, and Drand-derived times mutually agree within skew, so the committed $A.h$ is corroborated by the honest (by (H3b)) TEE and Drand readings and cannot be a fabricated-fresh height — and fresh_s : $\text{now}.h - A.h \leq \theta.\text{max_fs}$. A genuinely stale result ($\text{now}.h - A^*.h > \theta.\text{max_fs}$) therefore cannot clear; any C violating either check is **Rejected**, contradicting **Clear**. $\square$

The theorem is modest by design. Its content is not a new hardness result but a precise statement of *trust localization*: the only ways to settle a stale computation are to break H , to forge Ed25519, to defeat enclave attestation (H3a), or to corrupt an attested enclave’s clock (H3b).

9.5 The honest boundary. Two boundaries follow immediately, and stating them is the point of the formalization rather than a caveat appended to it.

Without (H3b), Theorem 1 fails — exactly where the threat model says it does. A compromised enclave observes the block height and Drand round at signing time and can echo consistent values as its own TEE timestamp; step (2) collapses because A is no longer guaranteed to equal A^* . Crucially this is not repaired by attestation: under TDXdown (§11) the enclave still passes (H3a) while violating (H3b). The triple anchor is therefore not the last line of defense against enclave compromise — and neither, on its own, is attestation; an explicit honest-clock assumption is required, exactly as stated in §7 constraint 6 and §11.

fresh_m and fresh_i are only as sound as their oracle. They rest on the canonical-state oracle (H4); the reference gate now enforces them against that oracle (§10), realized in mock and real forms. fresh_c , by contrast, is **not** a settlement-decidable delta: $\text{now}.h - A.h$ is fresh_s , not f_c (which names the compute→commit gap and would need a completion timestamp the commitment lacks). It is handled *structurally* — a commit-at-completion committer makes $f_c \equiv 0$ by construction, and a late submission of the signed result is caught by fresh_s (see §10.2) — rather than enforced as a gate predicate.

Proposition 2 (extension, conditional). *Add to (H1), (H2), (H3a), (H3b) the assumption (H4) that a canonical-state oracle supplies the true model root and input-world state at now. Then the full gate of §9.2 is FGSS-sound for all four freshness types. Without (H4), the $\text{fresh}_m/\text{fresh}_i$ guarantees reduce to the freshness of the oracle itself.*

Proposition 2 is the formal restatement of this paper’s thesis. The DeFi oracle-freshness problem is not merely an analogy for proof-of-context; under the model it is a load-bearing assumption the construction inherits. Discharging (H4) — building the canonical-state oracle and proving its freshness — is, together with the execution-context-root sufficiency question of §8, the substance of the construction that remains open.

9.6 Multi-party gating. The agent-economy deployment of §10.5 settles a *negotiated* trade between N parties, each presenting its own commitment C_k against a shared settlement-time state. Define the multi-party gate $G^N(\{C_k\}, \text{now}, \theta) = \bigwedge_{k=1}^N G(C_k, \text{now}, \theta)$: the trade clears iff every party’s commitment clears the single-party gate, and the win condition generalizes pointwise ($\mathcal{A}$ wins if $G^N = \text{Clear}$ while *some* party’s genuine context is stale).

Corollary 1 (multi-party FGSS). Under the assumptions of Theorem 1 (resp. Proposition 2), G^N is FGSS-sound: $\Pr[\mathcal{A} \text{ wins the } N\text{-party game}] \leq N \cdot \text{negl}(\lambda) = \text{negl}(\lambda)$. *Proof.* If $G^N = \text{Clear}$ then $G(C_k, \text{now}, \theta) = \text{Clear}$ for every k ; a win requires some party k^* whose genuine context is stale yet $G(C_{k^*}, \text{now}, \theta) = \text{Clear}$, which is exactly a single-party FGSS win for k^* . By Theorem 1 (resp. Proposition 2) each such event has probability $\leq \text{negl}(\lambda)$; a union bound over the N parties

preserves negligibility for any $N = \text{poly}(\lambda)$. $\square$ The reduction is deliberately trivial: multi-party gating buys no new soundness risk, because the per-party guarantee composes by conjunction. What it *adds* is per-party *attribution* — the gate returns which party (and which freshness type) failed — so the protocol can refund or slash the responsible agent rather than voiding the trade wholesale.

10 Reference Implementation

The model of §9 is not aspirational: its objects, predicates, and the gate are implemented in a reference crate, `proof-of-context` (v0.3.1, MIT OR Apache-2.0). The crate is pure Rust with `#![forbid(unsafe_code)]`; its default build depends only on `sha2` and `ed25519-dalek`, and the canonical-state oracles of Proposition 2 ship as opt-in feature-gated modules. This section reports what the implementation establishes and, as importantly, what it does not yet enforce.

10.1 What the crate implements. The ten-component execution-context root, the triple anchor, the signed commitment, the per-type freshness thresholds (one struct of seven parameters — three skew bounds and four horizons — with Base-mainnet and permissive-testnet presets), and the settlement gate are all present as the types named in §9. The cryptography is real: the root is a SHA-256 over the canonical preimage, and the committer produces a genuine Ed25519 signature over the signing digest d . A software-key mock backend stands in for the TEE-backed committer, with χ carrying a `MockSoftware` vendor tag; this is suitable for testing the trait surface and the rejection logic, not for economic settlement, where the mock attestation would be replaced by TDX-quote and H100-attestation verification (a Phase-3 task; deployment path in §12).

10.2 What it enforces, and what it does not. A strict reading of the gate is warranted, because the gap between “the four types are defined” and “the four types gate payment” is precisely the overstatement this program avoids. As of v0.3 the reference gate computes $G_{\text{impl}} = \text{Verify} \wedge \text{consistent} \wedge \text{fresh}_m \wedge \text{fresh}_i \wedge \text{fresh}_s$ — integrity plus **three of the four** freshness types. The pieces: (i) `Verify` — signature and attestation-chain validity. (ii) `consistent` — refined to **internal triple-anchor agreement**: the commit anchor’s TEE- and Drand-derived wall-times (and, under the live-anchor feature, the block-derived time) corroborate *each other* within skew. This is a property of A alone — it detects a tampered or desynced commit clock — and replaces the A -vs-now form of earlier drafts, which conflated the integrity check with the settlement horizon (that form at a tight skew makes fresh_s unreachable); §9.2 and Theorem 1 now carry this refinement, with the staleness guarantee resting on fresh_s and `consistent` foreclosing a fabricated-fresh anchor. (iii) fresh_s — the settlement window $\text{now}.h - A.h \leq \text{max_fs}$. (iv) fresh_m , fresh_i — decided against a canonical-state oracle (Proposition 2’s H4), supplied in three forms: a software *mock* for tests, and two **real**, feature-gated oracles — a witness-presented BaseOracle attestation verifier for fresh_i (the caller presents signed input attestations whose canonical-JSON root must match the committed input-manifest root), and an **M-of-N** committee/quorum model-lineage registry for fresh_m (publishers sign a canonical lineage; epoch distance is read from lineage position at the settlement anchor). The one type the gate does **not** enforce is fresh_c : $\text{now}.h - A.h$ is the commit→settle gap (that is fresh_s), **not** the compute→commit gap fresh_c names, which needs a computation-completed timestamp the commitment does not carry. (Earlier revisions called fresh_c “a block-height delta like fresh_s , simply not yet wired”; that was wrong — the delta *is* fresh_s .) fresh_c is handled *structurally*: a **commit-at-completion** committer signs at production, so $f_c \equiv 0$ by construction in-enclave, and a worker that sits on the *signed* result is caught by fresh_s ; an out-of-enclave measurement would require a TEE-attested completion timestamp (future work). The one disclosed

non-injectivity in the reference root is resolved as of crate v0.3.1: the `AttentionImpl::Other(tag)` discriminant previously folded an 8-bit tag into seven bits (`0x80 | (tag & 0x7F)`), so two `Other` tags differing only in the high bit produced the same root — exactly the trivial-evasion vector §8 warns of. `Other` now encodes as the two-byte sequence `0x80 || tag`, restoring injectivity; the enumerated `FlashAttention2/Sdpa/FlexAttention` variants keep their single-byte encodings, so roots committed with enumerated variants are unchanged, and no published test vector had used `Other`. Regression tests pin the previously-colliding pair. (The v0.2 backwards-clock short-circuit is likewise resolved: the gate accumulates every violated predicate.)

10.3 What the tests establish. The suite passes 44 tests on the default feature set and 84 with all oracle features enabled (live Drand and Base-RPC anchor tests are network-gated and excluded from those counts). It exercises these claims relevant to the model:

- *The root is sensitive to every in-scope component.* Perturbing the sampling seed, toggling the kv-cache, or switching the attention implementation or precision mode each changes the root. The last two matter specifically: attention-implementation and precision-mode are the attack-surface vectors attributed to TOPLOC [16], and the tests confirm they sit inside the commitment rather than outside it.
- *The rejection paths fire, per type.* A commit anchor whose clocks disagree internally beyond skew is `Rejected([Computational])`; a model the oracle reports beyond `max_fm` epochs (or off the canonical lineage) is `Rejected([Model])`; an input-world state lagging beyond `max_fi` is `Rejected([Input])`; a settlement beyond `max_fs` is `Rejected([Settlement])`. The result carries the *set* of violated types, and simultaneous violations are reported together; a flipped signature byte or a post-signing output-hash change is `InvalidSignature`.
- *The oracle binding holds.* A disclosed execution-context root that does not hash to the committed root is rejected before any field is read; an input attestation whose canonical-JSON root mismatches the committed input-manifest root is treated as stale; and an M-of-N model lineage is adopted only when a quorum of registered publishers signed the identical canonical snapshot.
- *The signed object round-trips.* A commitment serializes and deserializes without loss and re-verifies — the property the wire-format specification depends on — and the canonical-JSON hashing matches the cross-language test vectors (`test-vectors/v0.1.json`) byte-for-byte across the Rust, JavaScript, and TypeScript implementations.

10.4 Cost. The primitive is cheap in the sense §7 constraint 2 requires. The root is a single SHA-256 over a 251-byte preimage; a commitment carries 192 bytes of fixed cryptographic material (32-byte root, 32-byte anchor, 32-byte output hash, 64-byte signature, 32-byte key) plus the attestation chain; committing costs one hash and one Ed25519 signature, settling one verification. Measured on commodity hardware, the root hash runs in about 0.2 μ s, a commitment in about 11.5 μ s, and a verification in about 25 μ s.¹ The oracle paths are the same order of magnitude, dominated by Ed25519 verification: the canonical-JSON hash (shared by the f_i and f_m roots) runs in about 1.1 μ s, an f_i BaseOracle witness verification in about 26 μ s (one signature check), an f_m 2-of-3 quorum

¹AMD Ryzen 7 9800X3D, rustc 1.94.0, release profile, ed25519-dalek 2.x, 200,000 iterations per primitive via the `measure_costs` example in the reference crate (oracle-path figures with `-features "oracle-fi oracle-fm darkpool-sol"`). Figures indicate order of magnitude, not a benchmark claim against alternative constructions.

lineage adoption in about 53 μ s (two signature checks plus parsing), and the full multi-party dark-pool gate in about 25 μ s per party. At those costs the overhead of carrying a proof-of-context commitment is negligible against any non-trivial inference, which is the property the economics require.

10.5 A concrete on-chain instantiation: the agent-to-agent dark pool. The reference crate is venue-neutral; to show the primitive gating real settlement, it is instantiated on-chain in an agent-to-agent (A2A) perp dark pool — a market in which one agent posts an *intent* (a price band) and another posts a *response* (a price), and a negotiated trade clears only when both accept. This is precisely a setting where *proof-of-context* bites: each agent’s quoted price is the output of reasoning over a market context (the price/oracle state it observed), and a trade negotiated against stale context is economically toxic. The deployment adds, inside the dark pool’s settlement instruction, an `freshi gate` read trustlessly from chain state — the canonical market price’s last-update timestamp must be within a freshness budget of the settlement clock (`now − last_price_update ≤ budget`), filling a gap where the price was written but never checked at trade time — and binds to each quote a 32-byte **context commitment** (the hash of the off-chain context the agent reasoned over), authenticated by the agent’s own transaction signature. `freshs` is supplied natively by the intent/response expiry windows; `freshm`/`freshc` are deferred at this layer. The construction generalizes the single-commitment gate of §9 to a **multi-party** one (formalized as G^N in §9.6, with Corollary 1 reducing its soundness to a per-party application of Theorem 1): settlement clears only if *every* negotiating party’s quote was made against fresh context — the natural form for the agent economy, where the parties to a trade each carry their own context. The on-chain code is an engineering realization of G^N ; it is reported as evidence that the gate composes into a live settlement path rather than only a standalone verifier.

10.6 Evaluation. Three measurements ground the claims above.

- *On-chain cost.* A full negotiated dark-pool settlement — both parties’ commitments, the `freshi` price-freshness gate, the two `open_position` CPIs into the perp engine, and the vault transfers — consumes **120,799 compute units** on a local validator, about 8.6% of Solana’s 1.4M-CU per-transaction budget. The freshness gate itself adds only a market-PDA derivation, an 8-byte account read, and two integer comparisons; the total is dominated by the position-opening CPIs, not by the gate. Freshness gating is therefore affordable on the settlement hot path.
- *Test coverage.* The off-chain crate passes 84 tests across the default and all-oracle feature sets — exercising the per-type rejection paths, the oracle bindings (root mismatch, manifest mismatch, sub-quorum), the multi-party attribution, and the cross-language vectors. The on-chain dark-pool program passes a 96-test Anchor suite end-to-end on a local validator, including the `freshi`-gate stale-price rejection and the `fresh-price` clear.
- *Cross-implementation convergence.* The canonical-JSON SHA-256 construction produces byte-identical digests across the Rust, JavaScript, and TypeScript implementations on the shared test vectors (`test-vectors/v0.1.json`), now including the f_i input-manifest and f_m model-lineage roots — the empirical basis for the interoperability the wire format assumes.

These are engineering measurements on a single machine and a local validator, not a comparative benchmark; they establish feasibility and cost order-of-magnitude, which is what the economic argument of §7 requires.

11 Empirical Calibration of Triple-Anchor Thresholds

The triple-anchor slash threshold (§7 constraint 6) is a parameter that must be set with empirical grounding. Handwaving a number here breaks the protocol in two directions: too tight and honest workers get slashed under network load; too loose and the exploitable window swallows the protection. We present initial measurements and proposed defaults below; this section is intended to replace the “pending measurement” placeholder with defensible numbers.

Measurements (conducted 2026-04-22):

1. **Base L2 block-time distribution.** Sample of three non-overlapping ~ 500 -block windows from Base mainnet via `mainnet.base.org` JSON-RPC (`eth_getBlockByNumber`). Results:
 - Target block time: 2.0 seconds (post-June-2024 Base upgrade).
 - Mean under nominal conditions: **2.05 s** (1-day and 7-day windows); 98.1–99.2% of blocks hit the 2 s target exactly.
 - Median / P95 = 2 s; P99 = 2–6 s.
 - Observed tail event: a single 128-second sequencer gap within one of the 500-block windows — a real operational stall within the measurement run. Rolling mean under that event: 2.46 s. Standard deviation shifts from ~ 0.4 s (nominal) to ~ 6.6 s (during the stall).
 - Conclusion: ± 2 blocks absorbs P99 nominal variance; sequencer stalls are real and must be handled by a protocol-level retry/deferral path rather than by widening the slash threshold to include them.
2. **Drand mainnet cadence and propagation.** Chain information fetched from the public Cloudflare mirror (`drand.cloudflare.com`; the primary `api.drand.sh` endpoint returned HTTP 502 during measurement — itself a data point on CDN reliability). Results:
 - Period: **30 seconds** (deterministic by construction; round timestamp = genesis + period·round).
 - Chain hash: `8990e7a9aaed2ffed73dbd7092123d6f289930540d7651336225dc172e51b2ce`; scheme pedersen-bls-chained.
 - Post-emission availability latency: ≈ 1 second from a residential South American client to the Cloudflare mirror (HTTP RTT 45–411 ms, mean 154 ms over 6 samples).
 - Conclusion: ± 1 round (± 30 s) is well above measured propagation variance and leaves margin for fetcher-side mirror failures.
3. **TEE clock drift and attacker bounds (literature synthesis).** Benign drift on TSC-backed TEE clocks is bounded at **15 ppm $\approx$ 54 ms/hour** worst-case honest drift [32, 34]. Adversarial manipulation is more serious: packet-delay attacks against external trusted-time services accumulate multi-second error over minutes [32]; the TDXdown attack [33] demonstrated CPU-frequency-controlled manipulation of `rdtsc`-observed intervals on current-generation Intel TDX, effectively making in-enclave timing attacker-controlled even under a valid attestation chain, since attestation certifies launch-time code integrity, not the runtime timing channel. Intel’s `sgx_get_trusted_time` API was deprecated in SGX SDK 2.9 (2020) and has no drop-in replacement; since then, all trusted-time schemes for SGX/TDX build time externally via authenticated remote servers. Empirical in-enclave clock-read error is microsecond-scale (mean 69 μ s, max 170 μ s) [34].

- **Conclusion:** a ± 5 s TEE timestamp threshold is well above honest drift (sub-millisecond at minute scale); its role is detecting accidental desync, not adversarial manipulation. Crucially, neither this threshold nor the triple-anchor — nor attestation itself — defends against a TDxdown-class timing compromise: attestation certifies launch-time code integrity, not the runtime timing channel, so a TDxdown-affected enclave passes attestation while reporting an attacker-controlled timestamp. That residual trust is named explicitly as the honest-clock assumption (§9.4, (H3b)); the thresholds below are calibrated for that honest-clock regime, and the triple-anchor is explicitly *not* the last line of defense against enclave compromise.

Proposed default thresholds, empirically justified:

- **Block height:** ± 2 blocks (covers P99 under nominal conditions; sequencer stalls exceeding this are an operational failure mode handled by retry/deferral, not slash).
- **TEE timestamp:** ± 5 seconds (two orders of magnitude above the 15 ppm honest-drift bound for commitment windows of hours; conservative margin against transient network jitter; not a defense against TEE compromise).
- **Drand round:** ± 1 round (± 30 seconds) (covers post-emission CDN propagation delay and single-mirror outages; absorbs client-side fetch variance).

Thresholds are asymmetric because the three clocks have characteristic noise profiles at different scales. The defaults above are defensible for an initial deployment; production tuning should refine them with larger samples, per-subnet economic-risk adjustment, and measurement under conditions of elevated MEV or network stress. The raw JSON-RPC measurement data and the literature-derived TEE bounds are available as a companion artifact; this paper treats the thresholds as empirically-grounded initial values rather than handwaved parameters.

12 Author Background and Deployment Path

The framing in this paper was not derived in isolation. Three production artifacts from the author’s prior work (2025–2026) informed the intuitions that crystallized into proof-of-context, and one of them is the natural testbed for a first deployment. None is *prior art* in the academic sense — they are production software, not published constructions. They are relevant because they establish that the author approached this problem from an agent-infrastructure deployment angle before formalizing it.

TrustLayer (github.com/Jc-asastu/TrustLayer) is an agent-reputation infrastructure on Base L2 with x402 micropayments. Its reputation-scoring mechanism is a historical sketch of *contextually-appropriate behavior over time* — the dimension proof-of-context formalizes into a real-time settlement primitive rather than an after-the-fact scoring mechanism.

BaseOracle (github.com/Jc-asastu/BaseOracle) is a pay-per-query data oracle for autonomous AI agents on Base, priced via x402 micropayments. BaseOracle enforces contextual freshness on a per-query basis at the data layer and embeds it directly in the payment primitive. The structural insight it operationalized — that *freshness must be verified at pay-time, not discovered retroactively* — is the same discipline §7 constraint 8 imports from DeFi into the compute layer for proof-of-context.

SUR Protocol (github.com/Jc-asastu/sur-protocol) is a perpetual futures DEX with an A2A Dark Pool and agent-to-agent settlement via x402. SUR is the natural testbed for a first deployment

of proof-of-context: the settlement rail, agentic counterparty model, on-chain reputation layer, and micropayment infrastructure are already in place. Adding a PoC-gated settlement path for inference-priced trades — where the protocol refuses to settle an inference payment if the execution-context root is too stale relative to the A2A market state — is the smallest real-world test of the primitive. The first construction will be prototyped on SUR before being abstracted into a standalone implementation.

13 Conclusion

The decentralized-ML protocol stack has absorbed the lesson that computations must be verifiably correct. The federated-learning literature has absorbed the lesson that asynchronous staleness must be *scheduled and weighted*. Production inference networks have absorbed the lesson that activation fingerprints can detect model and prompt modification at scale [16]. Centralized-provider property-attestation frameworks have absorbed the lesson that phases can be composed under challenge-response anti-replay [17].

What has not yet happened, to our knowledge, is the unification at the decentralized-marketplace layer: treating context-appropriateness as a single composable settlement-gating primitive that spans training, inference, and prompt-cache; that has a clear structural precedent in the DeFi oracle-freshness problem; that gates *contribution settlement* via cryptographic temporal-validity on four distinct freshness types; and that admits existing primitives as fragment-level instances rather than replacing them.

An eleven-protocol review (main analysis of nine: Ritual, Atoma, Nillion, io.net, Aethir, Gensyn/Verde, OpenGradient, Prime Intellect, Inference Labs; with addenda on Supra and Hyperbolic) confirms that no production decentralized-ML protocol currently binds (**prompt**, **KV-cache state**, **model version**, **input manifest**, **timestamp**) into a single freshness-bound verifiable object whose expiration blocks settlement the way a DeFi contract refuses to clear against a stale Chainlink round. The matrix of which primitives each protocol supports is presented in Appendix A.

We publish this paper to (a) establish the term *proof-of-context*, the attestation-as-settlement distinguishing axis, the four-freshness-type decomposition, the execution-context-root scope, and the cross-domain DeFi analogy on the academic record (priority date 22 April 2026, the original position-paper publication; this is revision v0.9.1 of 5 July 2026); (b) situate the contribution honestly against the substantial existing work in staleness-aware federated learning [5–9], proof-of-learning robustness [1–3, 10], centralized-provider property attestation [17], inference-activation LSH [16], and cryptoeconomic-offensive staleness [19]; and (c) invite protocol teams, independent researchers, and cryptographic-primitive designers to begin treating proof-of-context as a first-class composable settlement primitive rather than a collection of phase-specific or differently-motivated concerns.

The analogy is clean: 2026 decentralized-ML is 2020 DeFi. The next three years of work on the context-verification layer will determine whether the field pays for its stale-oracle equivalents the same way DeFi did.

Acknowledgements

This paper was drafted and refined iteratively with Claude (Anthropic) as a research assistant: literature survey, argument pressure-testing, and critical feedback across multiple revision rounds.

The author is the sole intellectual author and is solely responsible for all claims, errors, and design choices. The collaboration is documented in the public narrative *Opus* (jc-asastu.github.io/opus).

The v0.5 revision absorbs substantive critique on the framing (reformulating the distinction from PAL^{*}M onto the attestation-as-settlement axis), the internal structure (decomposing freshness into four distinct types, reorganizing §6 accordingly), the threat model (replacing a too-strong “three orthogonal clocks” claim with an honest “triple-anchor under a valid TEE attestation chain” claim), the construction details (prospective-only root bumps, runtime-upgrade notice period, historical-weight data-availability requirement, execution-context root scope formalization with attribution to TOPLOC), and the academic presentation (removal of miscategorized “prior art” section, addition of protocol survey matrix, correction of reference-list errors in a prior v0.3.1 patch).

A Protocol Survey Matrix

Nine production or testnet decentralized-ML protocols reviewed along the verification primitives relevant to proof-of-context. Columns: **PoL** = proof-of-learning primitive; **zkML** = zero-knowledge ML inference proofs; **TEE** = trusted execution enclave attestations; **LSH** = inference-activation locality-sensitive hashing (TOPLOC-style); **KV-cache** = explicit KV-cache integrity primitive; f_m = cryptographic model-freshness gating; f_i = cryptographic input-freshness gating; **Settlement-gating** = protocol refuses payment on freshness violation rather than weighting.

Protocol	PoL	zkML	TEE	LSH	KV-cache	f_m	f_i	Settlement-gating
Ritual	✗	✓ (opt)	✓ (opt)	✗	✗	✗	✗	✗
Atoma	✗	✗	✓ (sampling)	✗	✗	✗	✗	✗
Nillion	✗	✗	MPC (alt)	✗	✗	✗	✗	✗
io.net	✗	✗	PoW (hw only)	✗	✗	✗	✗	✗
Aethir	✗	✗	✗	✗	✗	✗	✗	✗
Gensyn/Verde	✓	✗	✗	✗	✗	✗	✗	✗
OpenGradient	✗	✓ (opt)	✓ (opt)	✗	✗	✗	✗	✗
Prime Intellect	partial (rollouts)	✗	✓ (INTELLECT-2)	✓ (TOPLOC)	✗	✗	✗	✗
Inference Labs	✗	✓	✗ (EigenLayer)	✗	✗	✗	✗	✗

Legend. ✓ = primitive supported as a protocol-level mechanism. ✓ (opt) = selectable at configuration time; not a uniform guarantee across applications. ✗ = not present as a protocol primitive. *partial* = coverage limited to a sub-phase (e.g., Prime Intellect’s TOPLOC covers per-rollout inference integrity but the training side relies on operational fault-tolerance without cryptographic verification). *MPC (alt)* = alternative trust model via multi-party computation instead of TEE. *PoW (hw only)* = hardware-reality proof-of-work, orthogonal to ML-workload verification.

Addenda (protocols with narrower relevance, excluded from the matrix above):

- **Supra Threshold AI Oracles** [28] — BFT threshold signatures over AI oracle outputs. Oracle-framing match is closest to proof-of-context’s vocabulary, but the construction is quorum-signing without temporal-validity semantics.
- **Hyperbolic Proof of Sampling** — probabilistic spot-check primitive for inference correctness. Complementary rather than overlapping with proof-of-context.

Interpretation. No protocol in the main table supplies f_m , f_i , or settlement-gating as a cryptographic primitive. Prime Intellect (via TOPLOC) comes closest to coupling per-inference context-binding but does not carry freshness semantics or gate settlement. The composability claim of this paper — that proof-of-context sits on top of existing primitives rather than replacing them

— is consistent with the pattern in the matrix: each row contributes a column, none contributes the rightmost three.

License and Citation

This paper is released under CC BY 4.0. To cite:

Maisu, J. C. (2026). *Proof-of-Context: The Missing Verification Layer in Decentralized ML Protocols* (Version 0.9.1) [Position paper]. github.com/Jc-asastu/proof-of-context, 5 July 2026.

References

- [1] Jia, H., Yaghini, M., Choquette-Choo, C. A., Dullerud, N., Thudi, A., Chandrasekaran, V., & Papernot, N. (2021). *Proof-of-Learning: Definitions and Practice*. IEEE Symposium on Security and Privacy. arXiv:2103.05633.
- [2] Zhang, R., Liu, J., Ding, Y., Wu, Z., Wang, Q., & Ren, K. (2022). “Adversarial Examples” for *Proof-of-Learning*. IEEE Symposium on Security and Privacy 2022. arXiv:2108.09454.
- [3] Srivastava, M., Arora, S., & Boneh, D. (2024). *Optimistic Verifiable Training by Controlling Hardware Nondeterminism*. arXiv:2403.09603.
- [4] Peng, Z., Zhao, J., Wang, R., Liao, X., Lin, J., Liu, Y., Cao, J., Shi, Y., Yang, L., & Zhang, M. (2025). *A Survey of Zero-Knowledge Proof Based Verifiable Machine Learning*. arXiv:2502.18535.
- [5] Zhang, W., Gupta, S., Lian, X., & Liu, J. (2016). *Staleness-aware Async-SGD for Distributed Deep Learning*. IJCAI 2016. arXiv:1511.05950.
- [6] Chen, M., Mao, B., & Ma, T. (2021). *FedSA: A staleness-aware asynchronous Federated Learning algorithm with non-IID data*. Future Generation Computer Systems, vol. 120.
- [7] Ma, L., Liu, Y., Jia, S., Zhou, J., Hu, Y., & Xie, X. (2024). *Dynamic Staleness Control for Asynchronous Federated Learning in Decentralized Topology*. WASA 2024, Springer LNCS vol. 14998.
- [8] Lu, C., Sun, Y., Yang, Z., Chen, J., Yin, D., & Zhu, J. (2026). *FedPSA: Modeling Behavioral Staleness in Asynchronous Federated Learning*. arXiv:2602.15337.
- [9] Wilhelmi, F., Afraz, N., Guerra, E., & Dini, P. (2023). *The Implications of Decentralization in Blockchain Federated Learning: Evaluating the Impact of Model Staleness and Inconsistencies*. arXiv:2310.07471.
- [10] Fang, C., Jia, H., Thudi, A., Yaghini, M., Choquette-Choo, C. A., Dullerud, N., Chandrasekaran, V., & Papernot, N. (2022). *Proof-of-Learning is Currently More Broken Than You Think*. arXiv:2208.03567.
- [11] Xing, Z., Zhang, Z., Zhang, et al. (2023). *Zero-Knowledge Proof-based Verifiable Decentralized Machine Learning in Communication Network: A Comprehensive Survey*. arXiv:2310.14848.

- [12] Adams, H., Zinsmeister, N., Salem, M., Keefer, R., & Robinson, D. (2021). *Uniswap v3 Core*. Uniswap whitepaper, March 2021.
- [13] Zhou, L., Xiong, X., Ernstberger, J., Chaliasos, S., Wang, Z., Wang, Y., Qin, K., Wattenhofer, R., Song, D., & Gervais, A. (2022). *SoK: Decentralized Finance (DeFi) Attacks*. arXiv:2208.13035.
- [14] Lee, H., Lee, J., Jin, S., & Ko, M. (2025). *Verifiable Dropout: Turning Randomness into a Verifiable Claim*. arXiv:2512.22526.
- [15] Arun, A., St. Arnaud, T., Titov, A., Wilcox, B., Kolobaric, V., Brinkmann, M., Ersoy, O., Fielding, B., & Bonneau, J. (Gensyn). (2025). *Verde: Verification via Refereed Delegation for Machine Learning Programs*. arXiv:2502.19405.
- [16] Ong, J. M., Di Ferrante, M., Pazdera, A., Garner, R., Jaghouar, S., Basra, M., Ryabinin, M., & Hagemann, J. (Prime Intellect). (2025). *TOPLOC: A Locality Sensitive Hashing Scheme for Trustless Verifiable Inference*. arXiv:2501.16007.
- [17] Chantasantitam, T., Caulfield, T., Duddu, V., Gunn, L., & Asokan, N. (2026). *PAL**M*: Property Attestation for Large Generative Models*. arXiv:2601.16199.
- [18] Gupta, A. (2025). *Can AI Keep a Secret? Contextual Integrity Verification: A Provable Security Architecture for LLMs*. arXiv:2508.09288.
- [19] Rhodes, S., Rao, A. (Opentensor Foundation). (2024). *Weight Copying in Bittensor: A Working Paper*. docs.learnbittensor.org/papers/BT_Weight_Copier-29May2024.pdf.
- [20] Bruschi, P., Esposito, F., Gagliardoni, T., & Rizzini, F. (2025). *SoK: Verifiable Federated Learning*. IACR ePrint 2025/2296.
- [21] Balan, O., Learney, R., & Wood, G. (2025). *A Framework for Cryptographic Verifiability of End-to-End AI Pipelines*. arXiv:2503.22573.
- [22] Baser, O., Sadeghi, A., Wang, J., Alves, M., Kazemian, H., Kang, S., Chinchali, S., & Vishwanath, S. (2026). *TensorCommitments: A Lightweight Verifiable Inference for Language Models*. arXiv:2602.12630.
- [23] Chen, H., Zhou, R., Chan, Y.-H., Jiang, Z., Chen, X., & Ngai, E. C. H. (2025). *LiteChain: A Lightweight Blockchain for Verifiable and Scalable Federated Learning in Massive Edge Networks*. arXiv:2503.04140.
- [24] Xu, Z., Zhang, Y., Ge, X., Li, J., Hu, Z., Zhang, W., Li, Z., & Chen, K. (2026). *Securing Retrieval-Augmented Generation: A Taxonomy of Attacks, Defenses, and Future Directions*. arXiv:2604.08304.
- [25] Caldarelli, G. (2025). *Can Artificial Intelligence solve the blockchain oracle problem? Unpacking the Challenges and Possibilities*. arXiv:2507.02125.
- [26] Mouzouni, C. (2026). *Context Kubernetes: Declarative Orchestration of Enterprise Knowledge for Agentic AI Systems*. arXiv:2604.11623.
- [27] Zhang, J. (2026). *Right to History: A Sovereignty Kernel for Verifiable AI Agent Execution*. arXiv:2602.20214.

- [28] Supra Research. (2025). *Threshold AI Oracles: Verified AI for Event-Driven Web3*. supra.com/documents/Threshold_AI_Oracles_Supra.pdf, 22 May 2025.
- [29] Prime Intellect. (2025). *INTELLECT-2: A Reasoning Model Trained Through Globally Decentralized Reinforcement Learning*. arXiv:2505.07291.
- [30] Prime Intellect. (2025). *Prime Collective Communications Library — Technical Report*. arXiv:2505.14065.
- [31] OpenGradient Foundation. (2026). *OpenGradient: Decentralized Infrastructure for Verifiable AI Execution*. Whitepaper, March 2026. opengradient.foundation/whitepaper.
- [32] Bettinger, M., Ben Mokhtar, S., Felber, P., Rivière, E., Schiavoni, V., & Simonet-Boulogne, A. (2025). *TriHaRd: Higher Resilience for TEE Trusted Time*. arXiv:2512.10732. IEEE INFOCOM 2026.
- [33] Wilke, L., Sieck, F., & Eisenbarth, T. (2024). *TDXdown: Single-Stepping and Instruction Counting Attacks against Intel TDX*. ACM SIGSAC Conference on Computer and Communications Security (CCS 2024). DOI 10.1145/3658644.3690230.
- [34] Fernandez, G. P., Brito, A., & Fetzner, C. (2023). *Triad: Trusted Timestamps in Untrusted Environments*. arXiv:2311.06156.

Revision note

v0.9.1 (5 July 2026): Patch revision, no new claims. (i) Corrects stale in-text version references: the §13 publication line and the License-and-Citation block still cited v0.7 / 11 June 2026 after two revisions; both now carry the current version. (ii) Records the resolution of the §10.2 non-injectivity: crate v0.3.1 encodes `AttentionImpl::Other(tag)` as the two-byte sequence `0x80 || tag` (previously a seven-bit fold that collided tags differing only in the high bit); enumerated-variant roots are unchanged, no published vector had used `Other`, and regression tests pin the previously-colliding pair. §9.1’s preimage-size statement gains the corresponding one-byte note. (iii) The crate additionally attributes multi-party integrity failures to the failing party (`PartyIntegrityError` carrying the party role), aligning the implementation with the per-party attribution §9.6 describes; the header version line is condensed, with history deferred to this note.

v0.9 (21 June 2026): Adds an empirical evaluation and a multi-party soundness result. New §9.6 defines the multi-party gate G^N (a negotiated trade clears iff every party’s commitment clears) and **Corollary 1** proves it FGSS-sound by a union bound over the single-party guarantee of Theorem 1 — the formal backing for the dark-pool deployment of §10.5. New **§10.6 Evaluation** reports measured numbers: a full multi-party dark-pool settlement consumes 120,799 compute units ($\approx 8.6\%$ of Solana’s 1.4 M-CU per-transaction budget; the gate adds only a PDA derivation, an 8-byte read, and two comparisons, with the CPIs dominating); the off-chain oracle paths run in 1.1 μs (canonical hash) to 53 μs (2-of-3 quorum); the suites pass 84 crate tests and a 96-test on-chain Anchor suite; and the canonical-JSON f_i/f_m roots converge byte-identically across Rust/JS/TS. §10.4 is extended with the oracle-path costs. No claim is retracted.

v0.8 (20 June 2026): Brings §10 *Reference Implementation* up to the v0.3 crate and corrects two model $\leftrightarrow$ implementation discrepancies. The reference gate now enforces consistent + fresh_m + fresh_i + fresh_s — integrity plus three of the four freshness types — where $\text{fresh}_m/\text{fresh}_i$ are decided against the canonical-state oracle of Proposition 2 (H4), realized in two real, feature-gated forms:

a witness-presented BaseOracle attestation verifier for fresh_i , and an M-of-N committee/quorum model-lineage registry for fresh_m . Two corrections: (i) consistent is refined throughout §9 — the §9.2 definition, the §9.3 FGSS win condition, and Theorem 1 with its proof — to **internal triple-anchor agreement** (a property of the commit anchor alone), since the *A-vs-now* form conflated the integrity check with the settlement horizon and at a tight skew makes fresh_s unreachable; the staleness guarantee now rests on fresh_s , with consistent foreclosing a fabricated-fresh anchor (the proved soundness of the $\{\text{consistent}, \text{fresh}_s\}$ subset is preserved in substance, not retracted). (ii) the v0.7 claim that fresh_c “is a block-height delta like fresh_s , simply not yet wired” is **withdrawn as an overclaim** — now $h - A.h$ is the commit→settle gap (fresh_s), not the compute→commit gap fresh_c names; fresh_c is handled structurally via commit-at-completion and omitted from the gate. New §10.5 reports a multi-party on-chain instantiation in an agent-to-agent perp dark pool (a trustless fresh_i price-freshness gate plus a per-quote context commitment, generalizing the single-commitment gate to N parties), and cross-language test vectors for the f_i input-manifest and f_m model-lineage roots were added to `test-vectors/v0.1.json`, including the array pre-sort contract.

v0.7 (11 June 2026): Adds the formal core the framing invited. Two new sections. §9 *Formal Model and Soundness*: precise definitions of the execution-context root, triple anchor, commitment, and the four freshness predicates; the *freshness-gated settlement soundness* (FGSS) game; Theorem 1, soundness of the enforced subset (three-clock consistency and the settlement window) under collision-resistance, EUF-CMA, attestation soundness, and an explicit honest-clock assumption; and Proposition 2, the oracle-dependent extension to model- and input-freshness that makes the DeFi oracle-freshness analogy a load-bearing formal assumption. §10 *Reference Implementation*: the **proof-of-context** crate v0.2 (real SHA-256 and Ed25519), 33 passing tests, the explicit statement that the gate enforces two of the four freshness predicates today (model and input freshness await the canonical-state oracle), and measured primitive costs. Former §9 Empirical Calibration, §10 Author Background, and §11 Conclusion renumbered to §11, §12, §13; the in-text references in §7 were updated accordingly. Section references in revision notes for v0.6 and earlier use the section numbering in effect at that revision. No claim from v0.6 is retracted; the position-paper content is unchanged and now carries a construction it previously deferred. The formal sections were additionally hardened against an adversarial review pass that split the attestation assumption into attestation-soundness (H3a) and an honest-clock assumption (H3b), separated the anchor-consistency predicate from the computational-recency horizon, tightened the FGSS game so the genuine commit-time anchor is well-defined, corrected the in-document citation block to v0.7, and disclosed a non-injective `Other`-tag encoding in the reference root.

v0.6 (22 April 2026): Polish revision closing out the four items flagged in the v0.5 public-readiness review:

1. **Direct-verified flagged references [8] FedPSA, [22] TensorCommitments, [26] Context Kubernetes, and [18] CIV** against primary arxiv sources. All four resolve to real papers with correct metadata; the CIV title verbatim on arxiv is “Can AI Keep a Secret? Contextual Integrity Verification: A Provable Security Architecture for LLMs” (single paper by Aayush Gupta, arXiv:2508.09288) — confirming the v0.5 citation against the v0.3 version that had paraphrased the title.
2. **Author’s production work removed from numbered reference list.** The former reference [32] (“Author’s production work: github.com/Jc-asastu ...”) was a personal-repository URL, not a published artifact, which is non-standard in a numbered bibliography. The material moves entirely into §10 “Author Background and Deployment Path” as inline URL citations, reserving the numbered reference list for published work.

3. **§9 subsection 3 threat-model consistency sentence added.** v0.6 added a sentence connecting the threat model to §7 constraint 6 on the TEE-timing/attestation relationship. (*v0.7 erratum: that sentence’s original framing — that TDxdown-class attacks are “caught at the attestation layer” — was incorrect and is superseded. Attestation certifies launch-time code integrity, not the runtime timing channel, so a TDxdown-affected enclave passes attestation; that residual trust is carried by the explicit honest-clock assumption, §9.4 (H3b). The triple-anchor remains calibrated for the honest-clock regime and is explicitly not the last line of defense against enclave compromise. See the corrected §7 constraint 6, §9.4–§9.5, and §11.*)
4. **TEE-timing references renumbered** [32] TriHaRd, [33] TDxdown, [34] Triad (previously [33][34][35]) consequent to removing the author-production-work reference. In-text citations in §9 updated accordingly.

No substantive change to arguments, conclusions, or structure from v0.5. The contribution remains as stated in prior revision notes. With these four items closed, the paper is intended to be ready for public sharing or submission as companion material to a technical application.

v0.5 (22 April 2026): Revision from v0.4 absorbing academic-presentation critique:

1. **§7 “Prior Art from the Author’s Portfolio” removed.** “Prior art” has specific technical meaning in academic writing — prior published work establishing precedent — which author’s production software does not satisfy. TrustLayer, BaseOracle, and SUR Protocol redistributed into new §10 “Author Background and Deployment Path” (author background informing the framing, plus SUR as deployment testbed), with a specific footnote-style attribution in §7 constraint 8 for BaseOracle’s “freshness-at-pay-time” insight.
2. **Acknowledgements reworted.** Removed “co-author” designation for LLM (no venue accepts this; the word damages credibility with any technical reviewer) and “parallel review process” phrasing (implies peer review that did not occur). Replaced with honest description: research-assistant role, iterative drafting with critical feedback, sole intellectual authorship attributed to the human author.
3. **§5 DeFi attacks citation added.** The “documented value losses” claim now cites Zhou et al.’s SoK on DeFi Attacks (arXiv:2208.13035) [13] rather than asserting unsourced figures.
4. **Appendix A “Protocol Survey Matrix” added.** The nine-protocol survey referenced in §11 is now presented as a concrete matrix (9 rows $\times$ 8 columns) with two addenda (Supra and Hyperbolic) listed separately. The earlier “nine-protocol survey ... with notes on Supra and Hyperbolic” was under-specified on count; the revision uses “eleven-protocol review (main analysis of nine; addenda on two)” and presents the matrix explicitly.
5. **Keywords reduced from 14 to 6** (proof-of-context · decentralized machine learning · oracle freshness · settlement gating · prompt-cache integrity · cryptoeconomic design), matching academic convention.
6. **References [28] INTELLECT-2 and PCCL separated** into two distinct references [29] and [30], eliminating the non-standard “See also:” combined format.
7. **§6 term-drift cleanup.** Unified “model-version freshness” and “model freshness” to f_m throughout; matching unifications for f_c , f_i , f_s .

8. **§9 renamed and populated with real measurements.** Renamed from “Expected-Skew Calibration (Experimental Roadmap)” to “Empirical Calibration of Triple-Anchor Thresholds”. Measurements conducted 2026-04-22 replace the “pending” placeholders: Base L2 block-time distribution sampled across three 500-block windows via public RPC (target 2 s, mean 2.05 s nominal, P99 2–6 s, observed tail 128 s stall); Drand mainnet cadence and propagation latency measured via Cloudflare mirror (30 s deterministic period, ≈ 1 s CDN propagation); TEE honest drift and attacker bounds synthesized from published literature (15 ppm TSC drift, multi-second adversarial via packet-delay attacks, unbounded via TDXdown). Defaults of ± 2 blocks / ± 5 s / ± 1 round are now empirically justified rather than handwaved. Three new references added: TriHaRd [32], TDXdown [33], Triad [34].

No substantive change to arguments or conclusions from v0.4. The contribution remains: naming the gap, distinguishing it from closest prior art on the attestation-as-settlement axis, decomposing freshness into its four types, formalizing the commitment scope, sketching the constraints on a construction, and inviting protocol teams to build it.

v0.4 (22 April 2026): Substantive revision from v0.3.1 reformulating the distinction from PAL^{*}M onto the attestation-as-verification vs. attestation-as-settlement axis, decomposing freshness into four types, formalizing the execution-context root scope, and replacing an over-strong triple-anchor claim with the honest version gated on a valid TEE attestation chain. (Also introduced §10 “Prior Art from the Author’s Portfolio” which v0.5 later redistributed — see above.)

v0.3.1 (22 April 2026, reference corrections patch): Reference corrections after primary-source audit. Commit e4fd121. Two fabricated references removed, fifteen titles corrected to verbatim arxiv.org titles, four author lists corrected, TOPLOC venue claim corrected to arXiv-only, Supra date corrected to May 2025.

v0.3 (22 April 2026): Phase 2 literature survey findings. §2 Background expanded from three primitives to five. §3 Related Work restructured with eleven subsections. References expanded from 13 to 31.

v0.2 (21 April 2026): Added Related Work section and tightened claims.

v0.1 (21 April 2026): Initial publication establishing the term.